



- **Parallelism Strategies**
- Speculative Decoding
 - Multiple-Token Prediction (MTP): EAGLE-1,2,3
 - Diffusion-LLM Based MTP: DFlash, DSpark
 - Beyond Speculative Decoding





- **Data Parallelism**
Split across batch dimension
- **Tensor Parallelism**
Split weight/activation matrix
- **Pipeline Parallelism**
Split across model layers
- **Expert Parallelism**
Split across experts (MoE)
- **Context Parallelism**
Split across sequence (Ring)

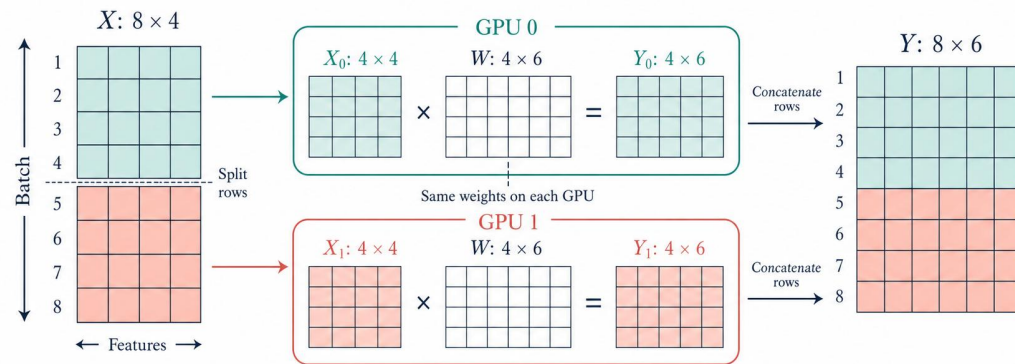


Fig. 1 DP

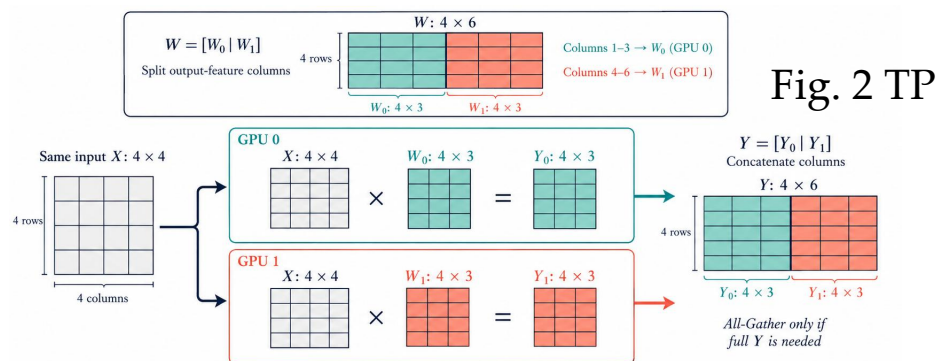


Fig. 2 TP

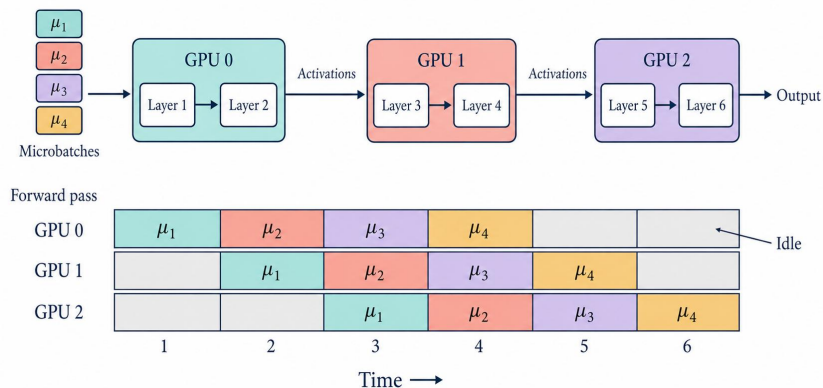


Fig. 3 PP

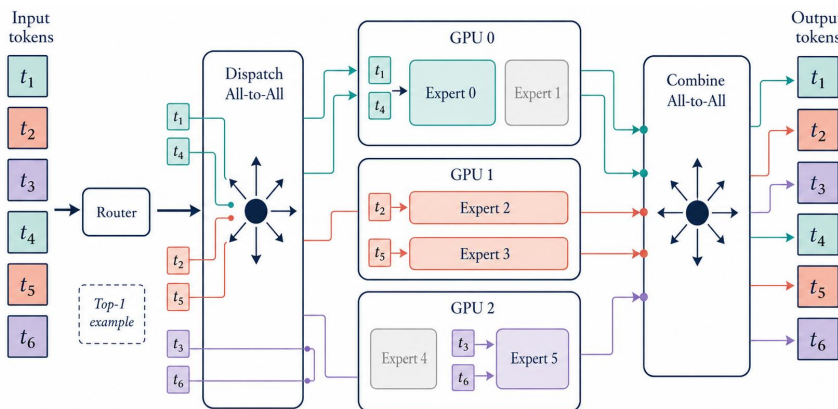


Fig. 4 EP

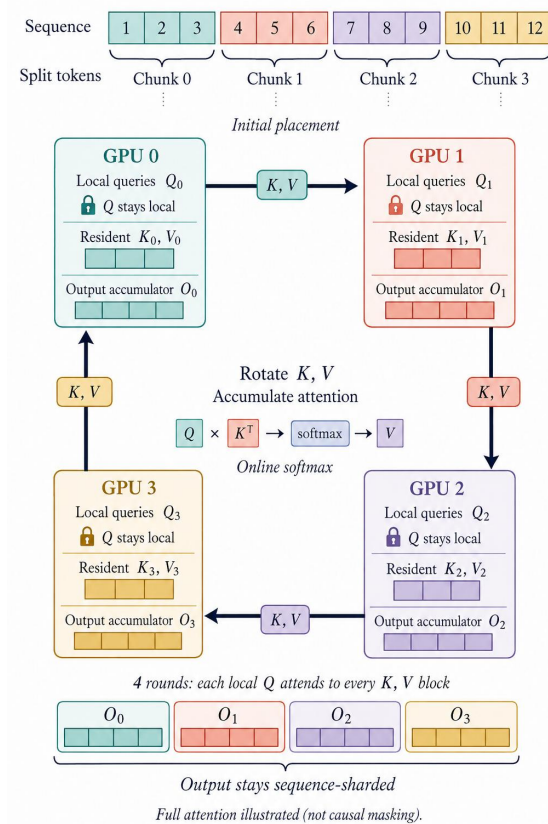


Fig. 5 CP





- Parallelism Strategies
- **Speculative Decoding**
 - Multiple-Token Prediction (MTP): EAGLE-1,2,3
 - Diffusion-LLM Based MTP: DFlash, DSpark
 - Beyond Speculative Decoding

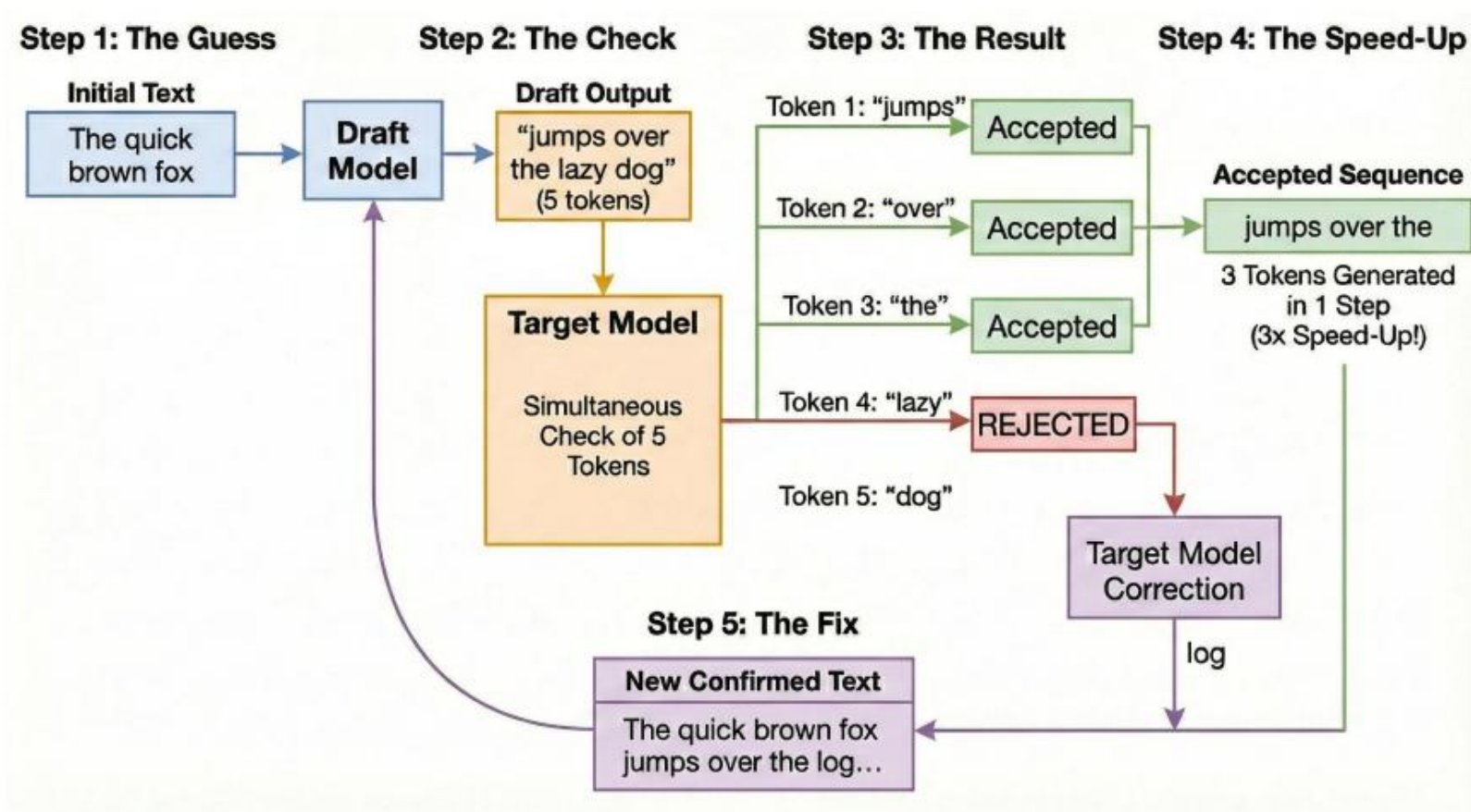




Speculative Decoding

Motivation

A target model can score all positions of the drafted sequence in parallel (**prefilling**), although the draft itself may be sequential (**decoding**).



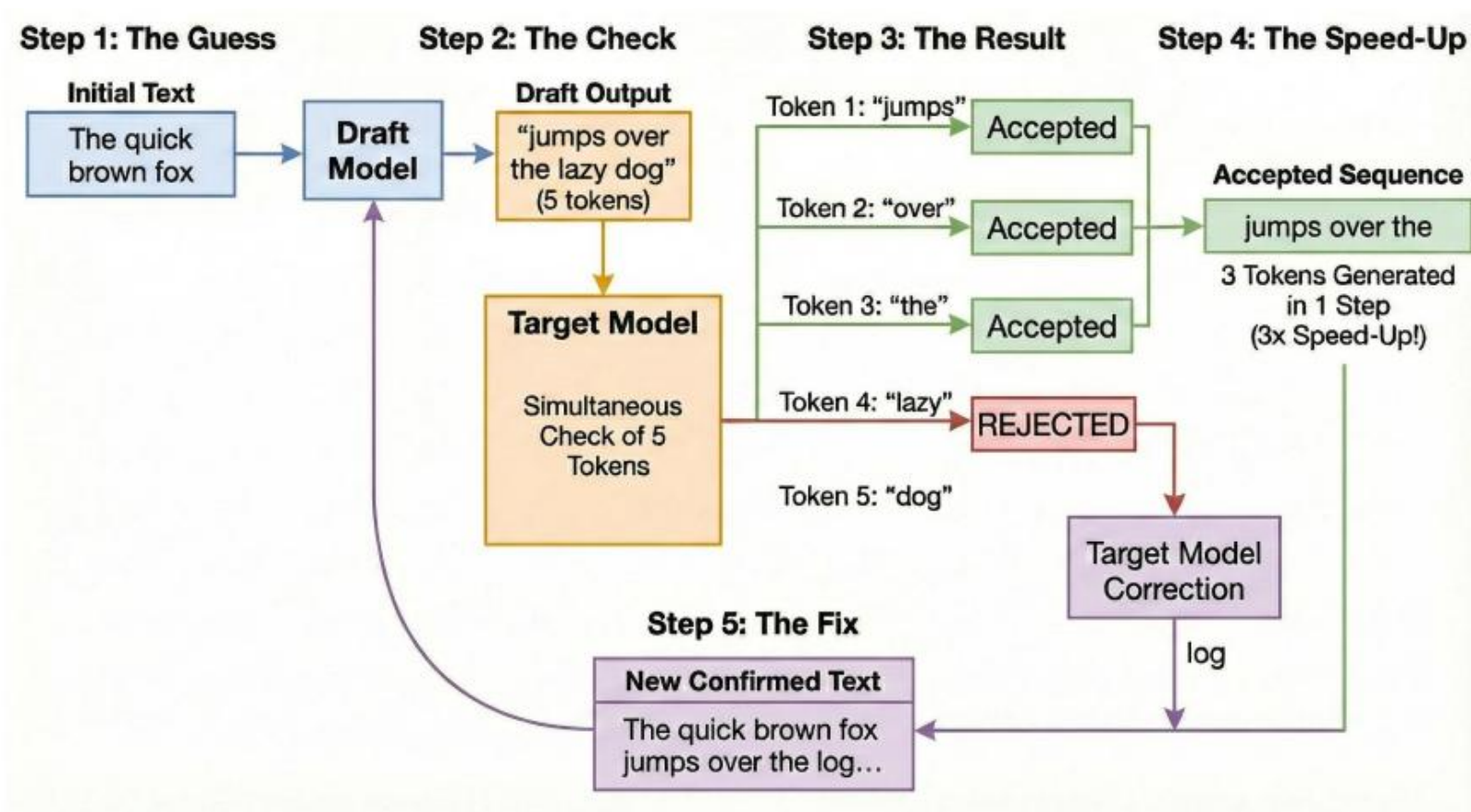
- **Prefilling**: process the entire input prompt in parallel to build the initial hidden states and KV cache before token generation begins (**compute-bound**)
- **Decoding**: generate output tokens autoregressively, one step at a time, reusing the KV cache from previous tokens (**memory-bound**)





Limitation

- Requires an **additional** draft model, and **draft–target mismatch** limits acceptance rate;
- Extra memory footprint, and decoding is always **memory-bandwidth** bound;



Four parts:

- ① Draft generation
- ② Candidate organization
- ③ Target verification
- ④ Acceptance / correction





- Parallelism Strategies
- Speculative Decoding
 - **Multiple-Token Prediction (MTP): EAGLE-1,2,3**
 - Diffusion-LLM Based MTP: DFlash, DSpark
 - Beyond Speculative Decoding



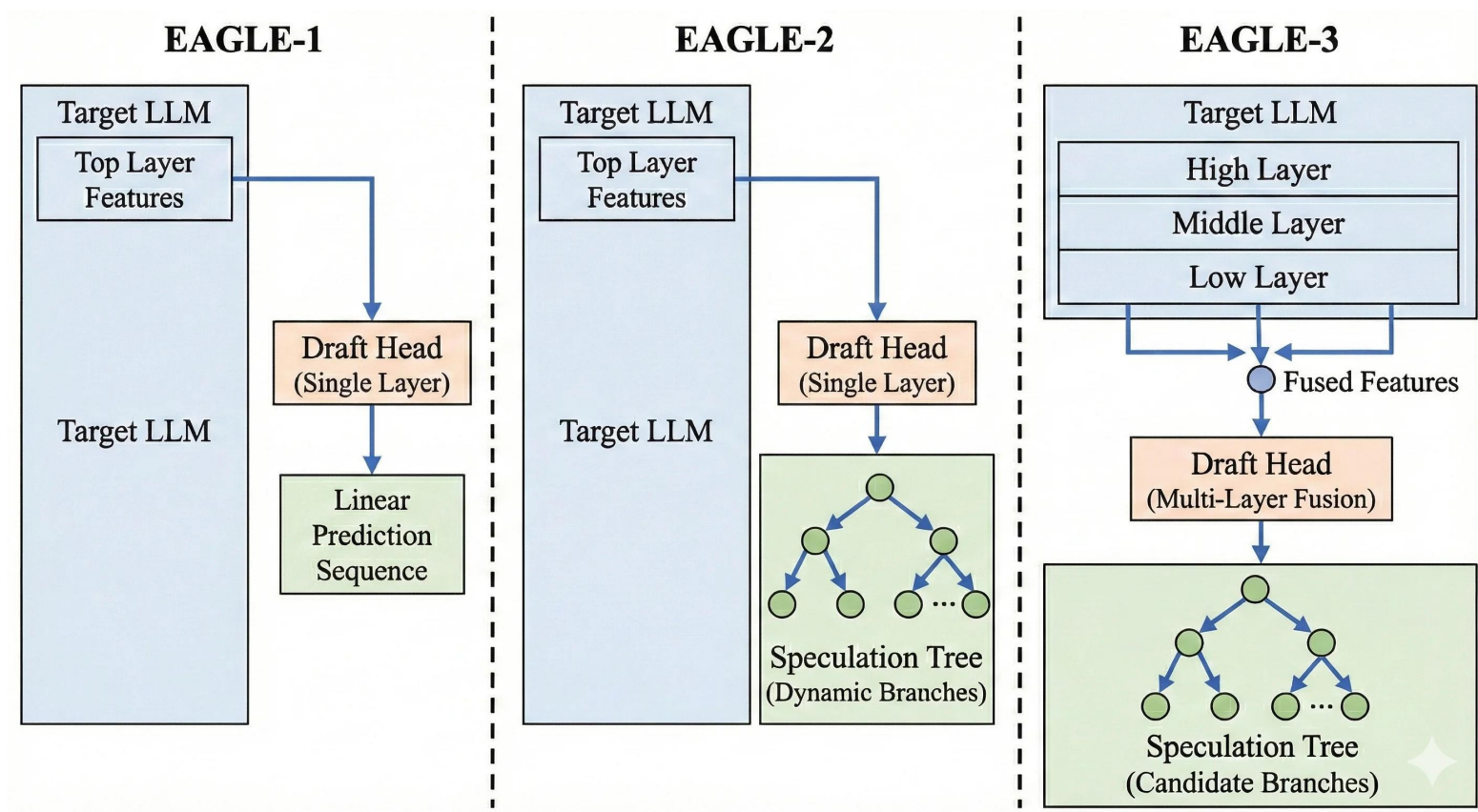


Multiple-Token Prediction: EAGLE-1,2,3

EAGLE: predict target-model **features** instead of directly drafting tokens;

EAGLE-2: dynamically allocates **draft-tree** branches according to predicted acceptance probabilities;

EAGLE-3: leverage **multi-level** target-model features for more accurate feature prediction.



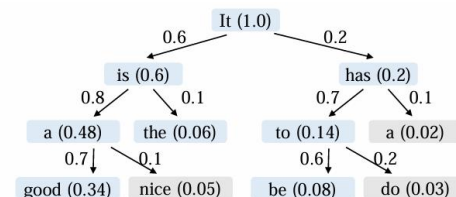
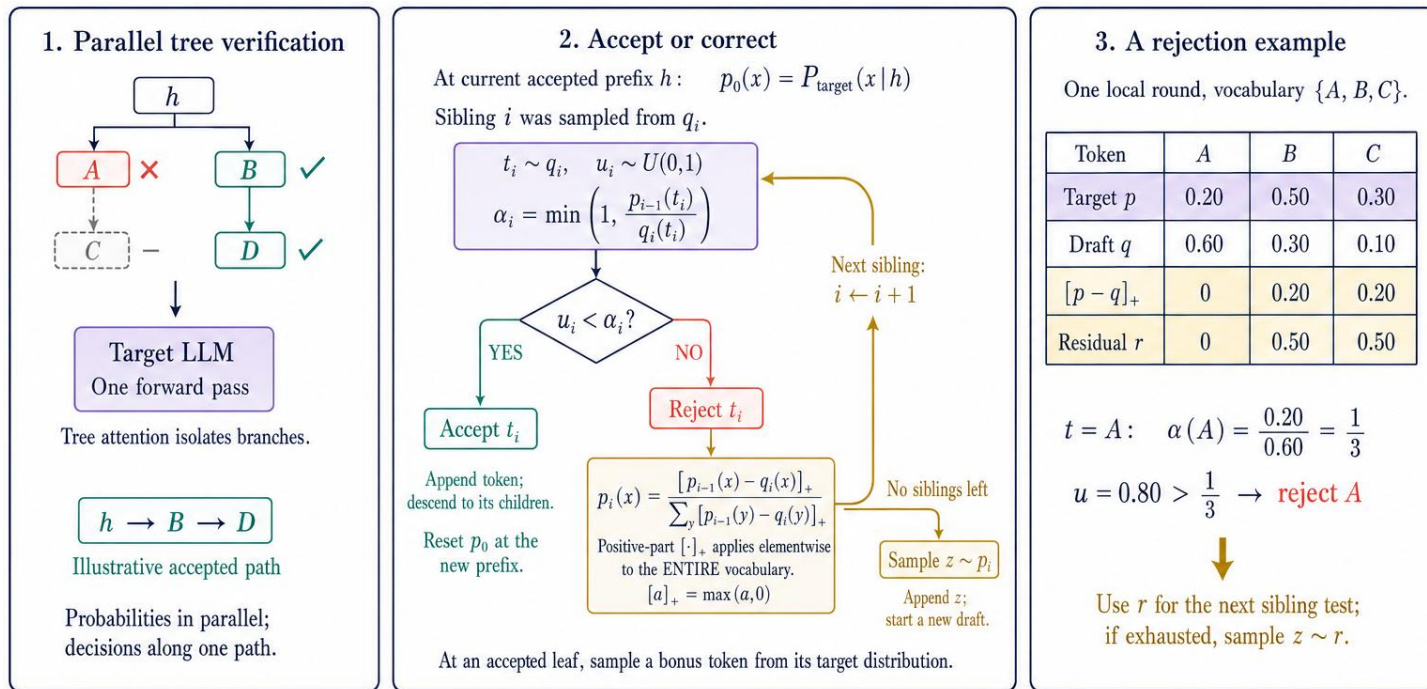
Multiple-Token Prediction: EAGLE-1,2,3

Vanilla Speculative decoding: $x_t \rightarrow \text{small LLM} \rightarrow x_{t+1} \rightarrow x_{t+2} \rightarrow \dots$

EAGLE: $(h_t, x_t) \xrightarrow{\text{EAGLE drafter}} \hat{h}_{t+1} \xrightarrow{\text{LM head}} \hat{x}_{t+1}$ fixed tree shape;

EAGLE-2: dynamic draft tree & tree attention.

Lossless Verification & Resampling:



Flatten to 1D

	It	is	has	a	the	to	good	be
Attention mask	It	is	has	a	the	to	good	be
It	✓							
is	✓	✓						
has	✓		✓					
a	✓	✓		✓				
the	✓	✓			✓			
to	✓		✓			✓		
good	✓	✓		✓			✓	
be	✓		✓			✓		✓

draft tree construction determines the **priority** of token candidates

Accept the overlapping probability mass between the draft and target distributions, and upon rejection, resample from the target's remaining probability mass, thereby exactly **preserving the target-model distribution**

Why the output distribution is preserved

$$\alpha(x) = \min\left(1, \frac{p(x)}{q(x)}\right), \quad Z = 1 - \sum_x \min(p(x), q(x)), \quad r(x) = \frac{[p(x) - q(x)]_+}{Z}$$

$$\mathbb{P}(\text{output} = x) = q(x)\alpha(x) + Zr(x) = \min(p(x), q(x)) + [p(x) - q(x)]_+ = p(x)$$

Apply this correction recursively across siblings and token positions.

q_i must match the actual proposal law. If $Z = 0$, rejection has probability zero.





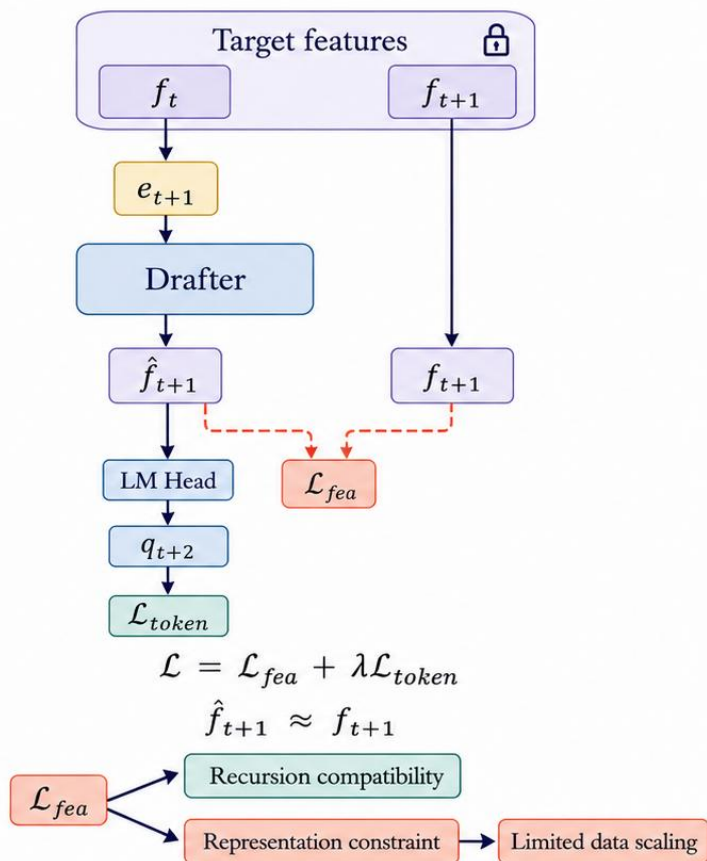
Multiple-Token Prediction: EAGLE-1,2,3

EAGLE-1/2 training loss: **feature-level** + **token-level**

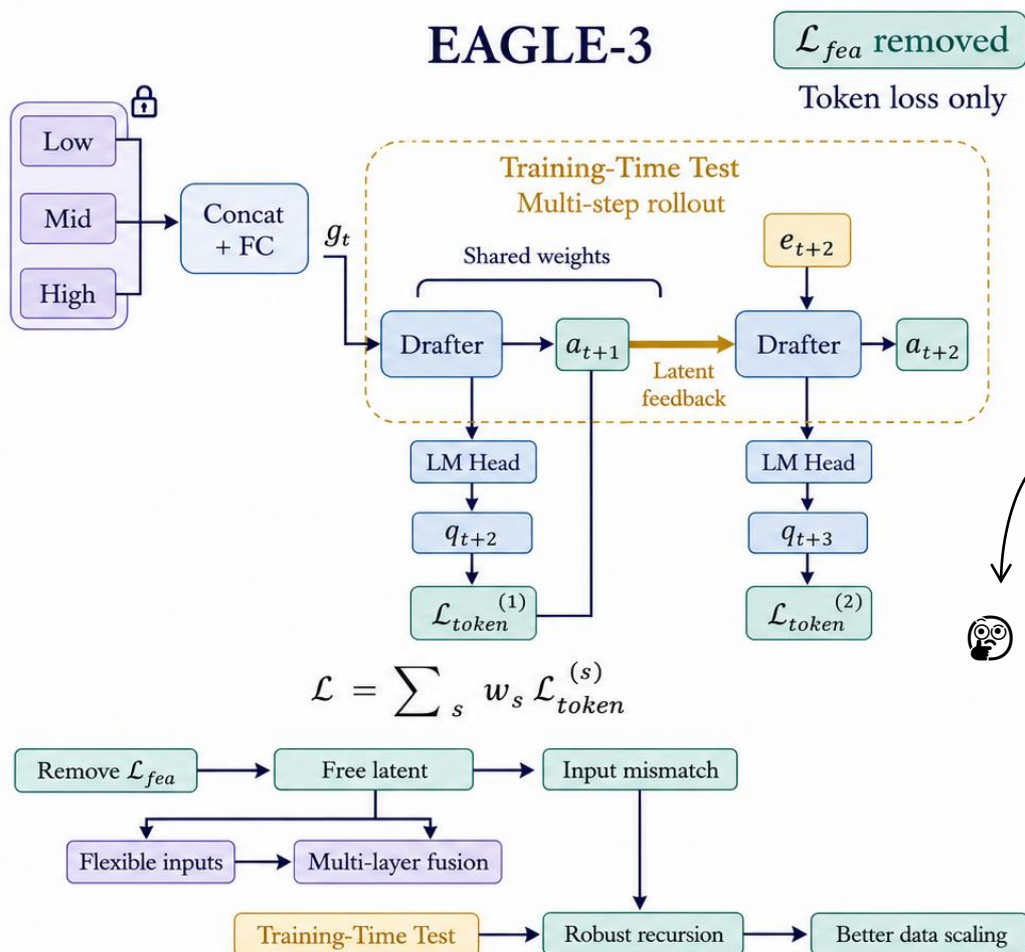
🧠 Goodhart effect

EAGLE-3 removes the feature-prediction constraint and improves drafting with **direct token prediction**, **multi-level** target-feature fusion, and **training-time test**.

EAGLE-1 / EAGLE-2



EAGLE-3



Remove feature regression, free the draft latent space and directly optimize token prediction;

Self-generated latent causes **train-test mismatch**, thus introducing training-time test, now multi-layer feature fusion becomes possible

🧠 [*When Hidden States Drift: Can KV Caches Rescue Long-Range Speculative Decoding?](#)





- Parallelism Strategies
- Speculative Decoding
 - Multiple-Token Prediction (MTP): EAGLE-1,2,3
 - **Diffusion-LLM Based MTP: DFlash, DSpark**
 - Beyond Speculative Decoding

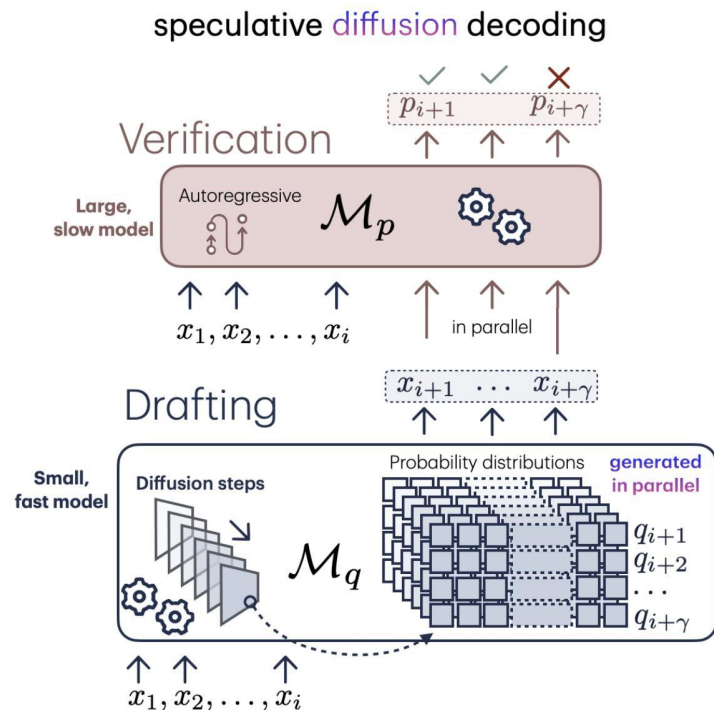
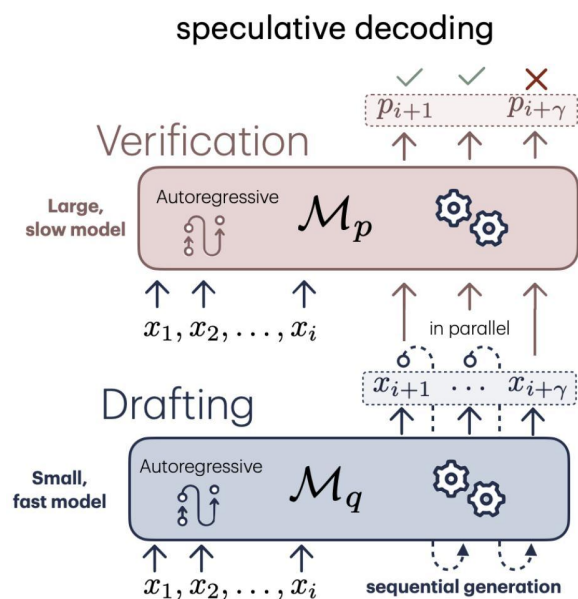




EAGLE family proposed better **drafter**, yet drafting remained **sequential**.

DFlash removes the autoregressive bottleneck of speculative drafting by generating a **block** of draft tokens in parallel with a **diffusion** model.

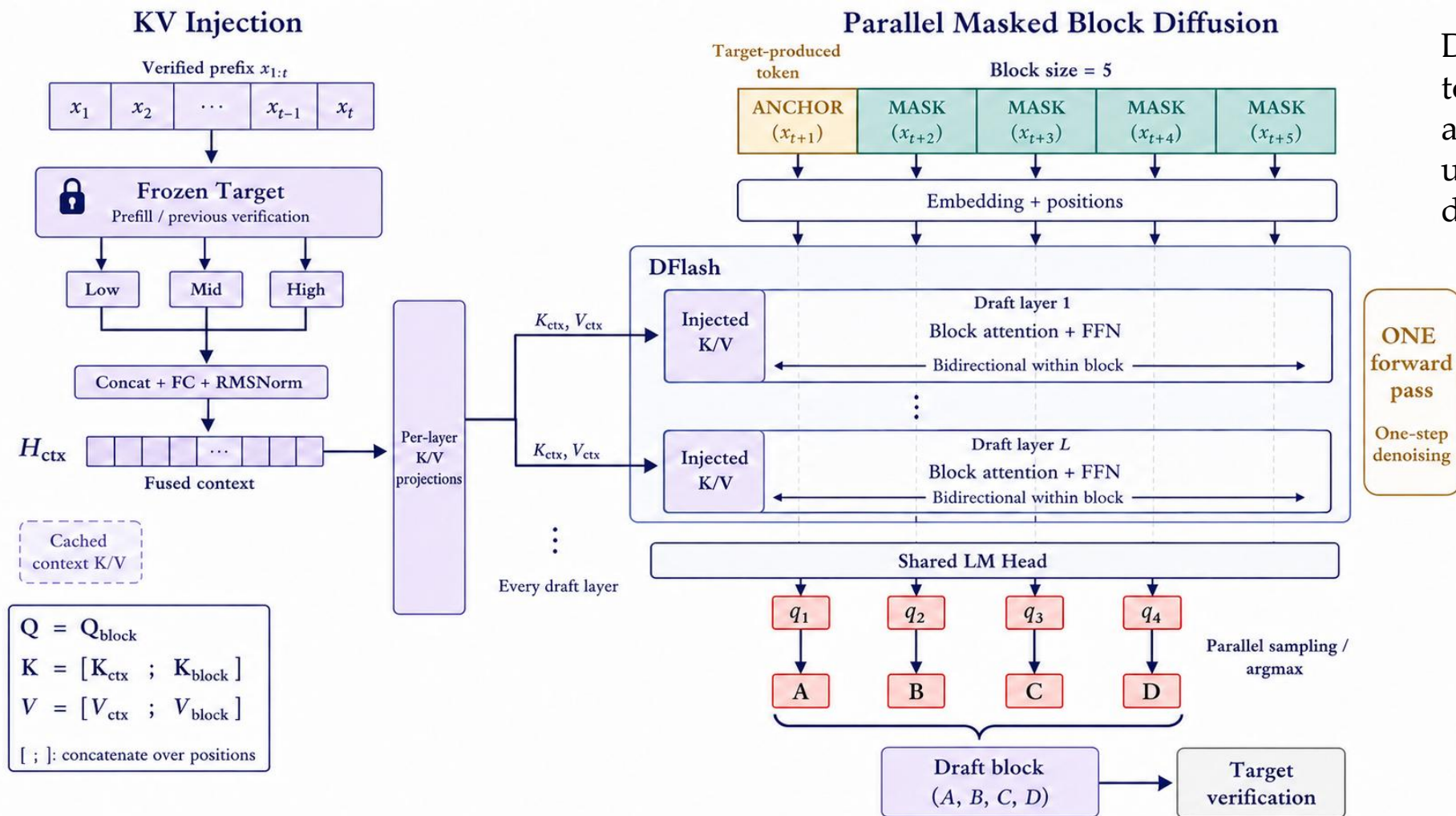
DSpark further introduces lightweight **intra-block** dependency modeling and **confidence-aware** verification to increase accepted prefix length while preserving parallelism.





Diffusion-LLM Based MTP: DFLash

DFlash accelerates inference by injecting **cached target-model features** into **every draft layer** and replacing autoregressive token-by-token drafting with a **single parallel masked-block** forward pass



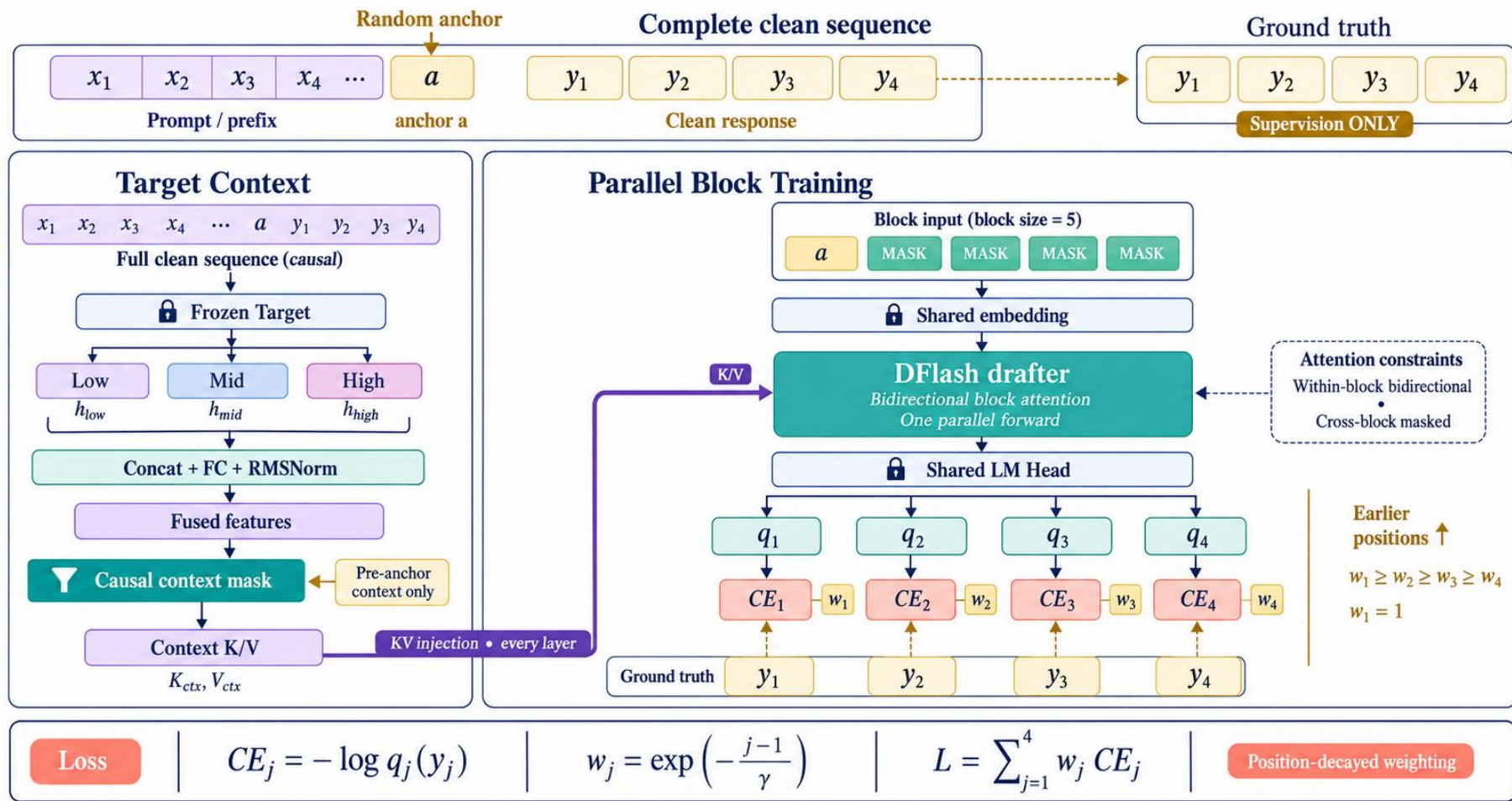
Diffusion determines how tokens are denoised, while attention is the mechanism used by the Transformer denoiser





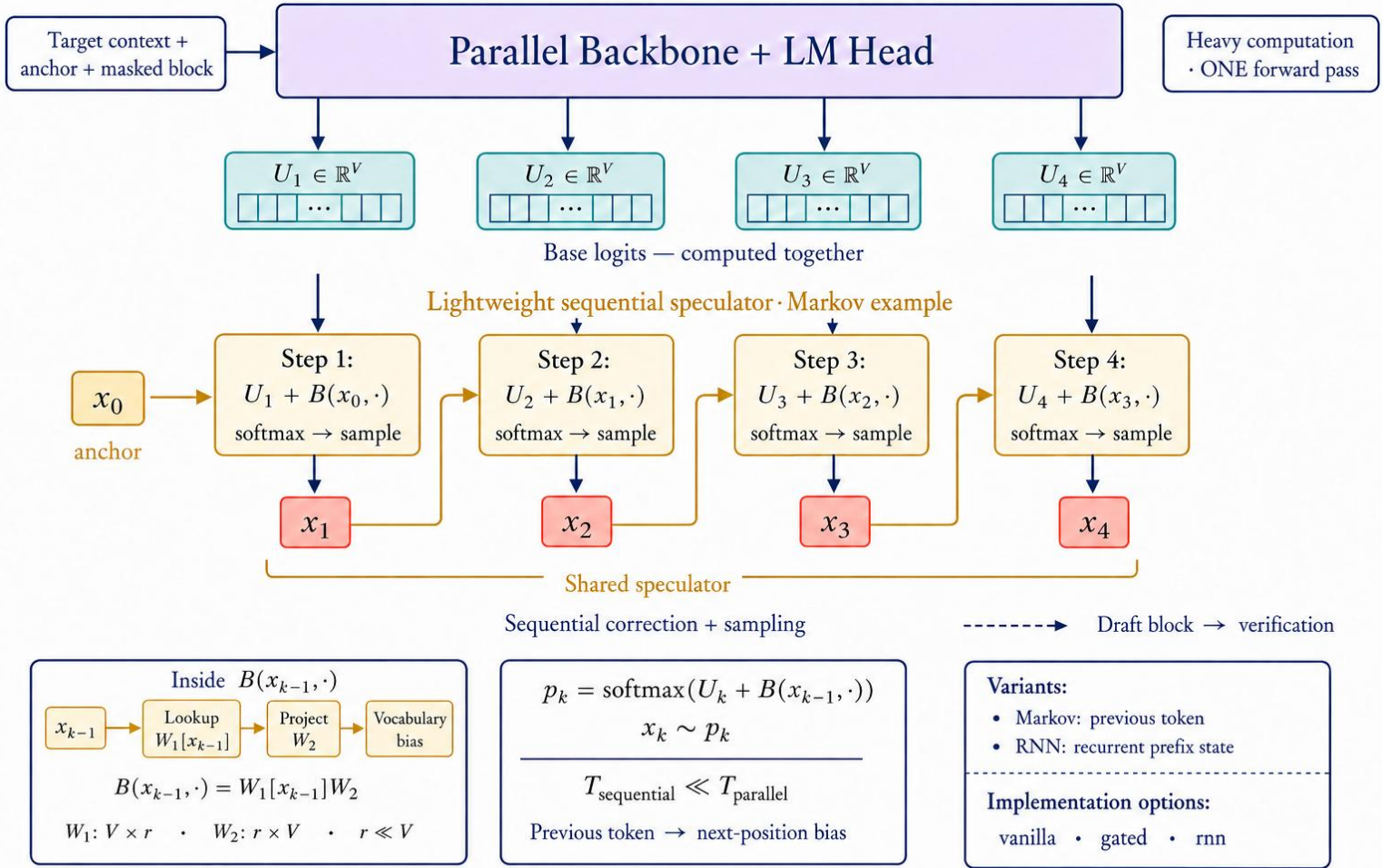
Diffusion-LLM Based MTP: DFLash

DFlash trains a lightweight diffusion drafter on **randomly anchored** masked blocks, conditioned on frozen target-model features via KV injection, using position-decayed cross-entropy to prioritize early speculative tokens.



DSpark aims to retain DFlash’s parallel drafting efficiency while recovering **token-to-token consistency** and **avoiding costly verification** of low-confidence suffix tokens.

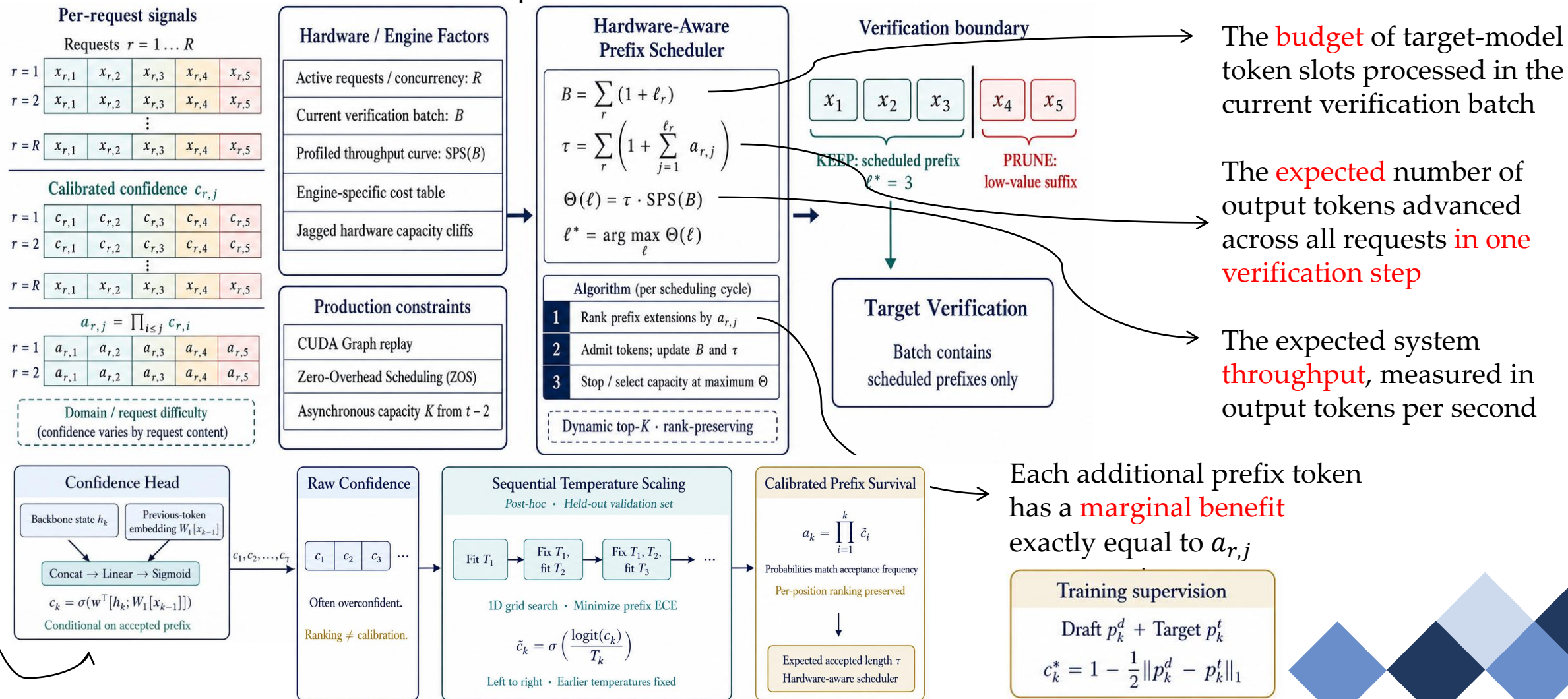
Lightweight causal dependency:





Diffusion-LLM Based MTP: DSpark

DSpark predicts the **survival probability** of each draft prefix and uses a **hardware-aware** scheduler to verify only prefixes whose expected acceptance gain justifies their target-model batch cost, pruning low-confidence suffixes before expensive verification.





- Parallelism Strategies
- Speculative Decoding
 - Multiple-Token Prediction (MTP): EAGLE-1,2,3
 - Diffusion-LLM Based MTP: DFlash, DSpark
 - **Beyond Speculative Decoding**

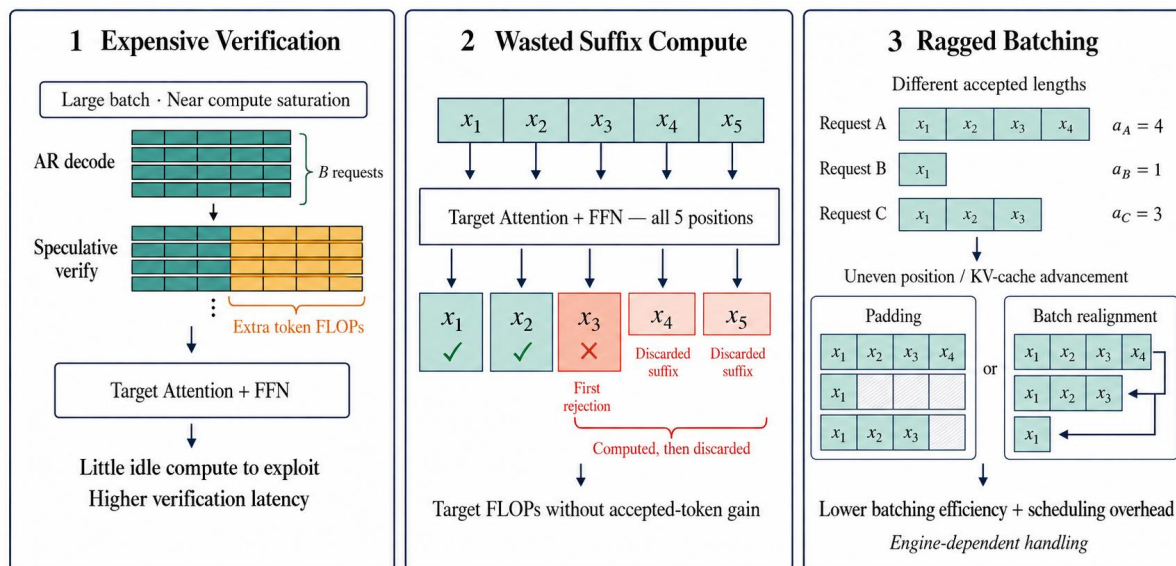




Why Existing Approach Struggle in Batch Serving

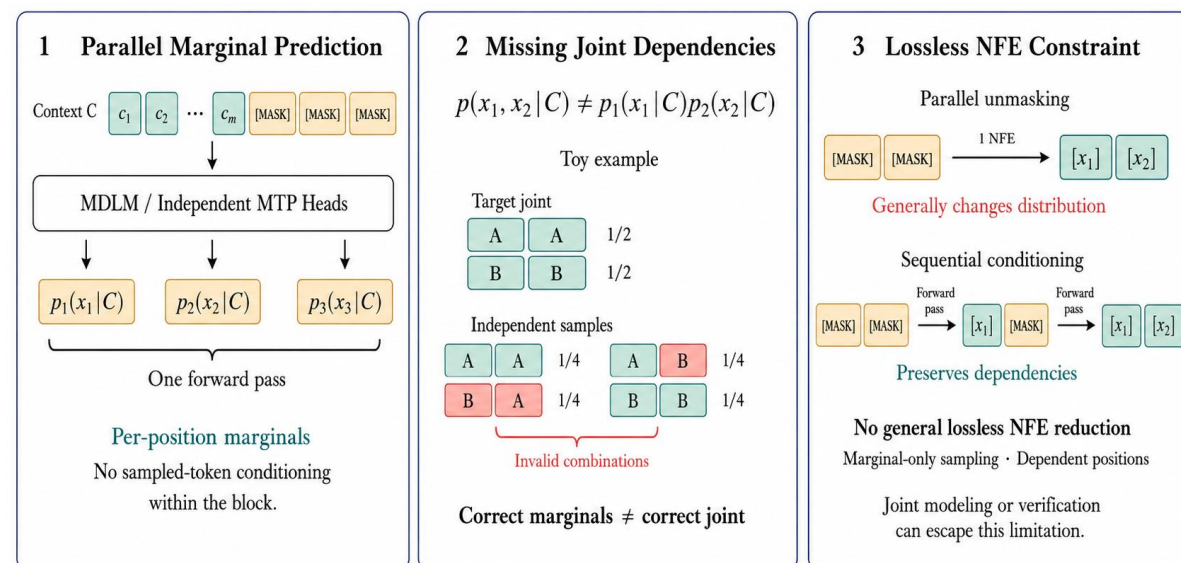
Draft-then-Verify Methods:

- Verification becomes expensive
- Rejected tokens waste substantial compute
- Variable acceptance creates ragged batching



Diffusion Language Models and MTP:

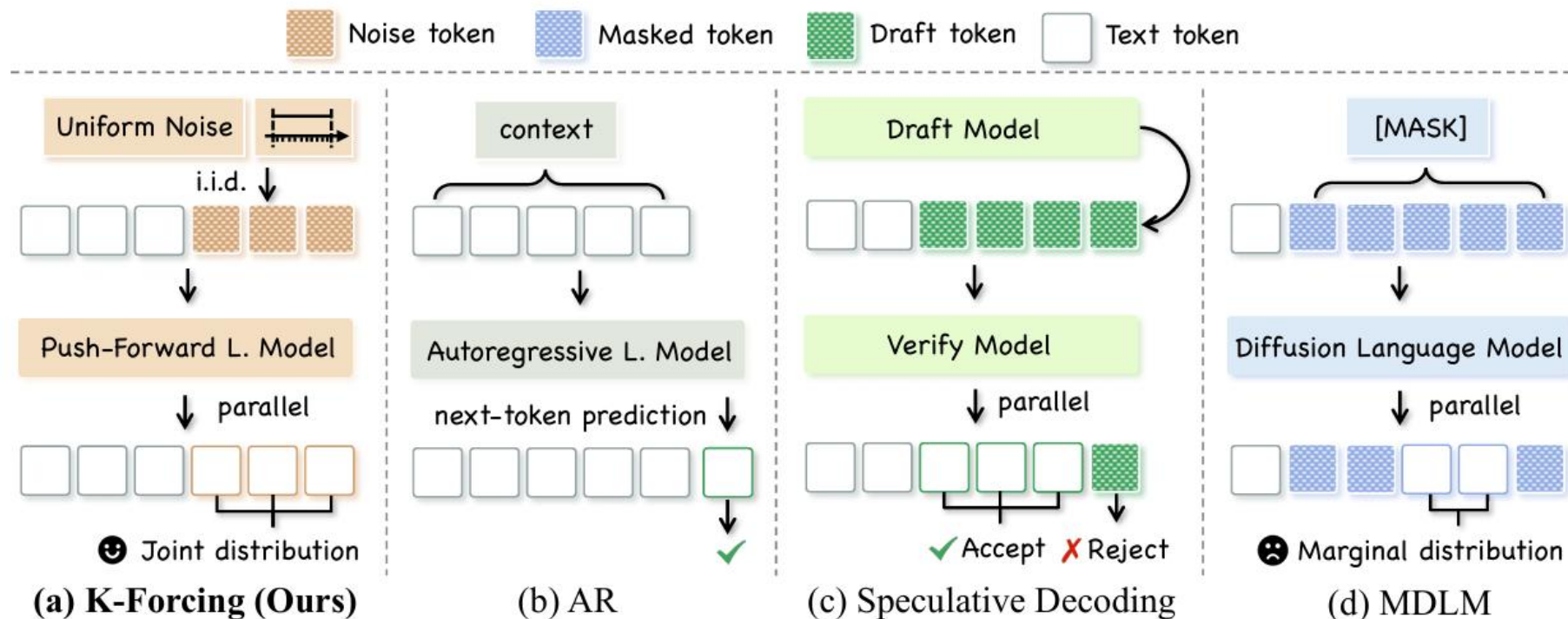
can predict multiple future positions in parallel, but they model **per-position marginals** rather than the joint future-token distribution, so parallel sampling generally breaks token dependencies and cannot reduce NFEs losslessly





K-forcing: Next-K-Token Decoding for Language Model

Observation: (b) AR leads to memory-bound decoding. (c) Speculative decoding yields a variable number of accepted tokens that breaks regular batching. (d) MDLM predicts masked positions in parallel from per-position marginals, rather than their joint distribution.

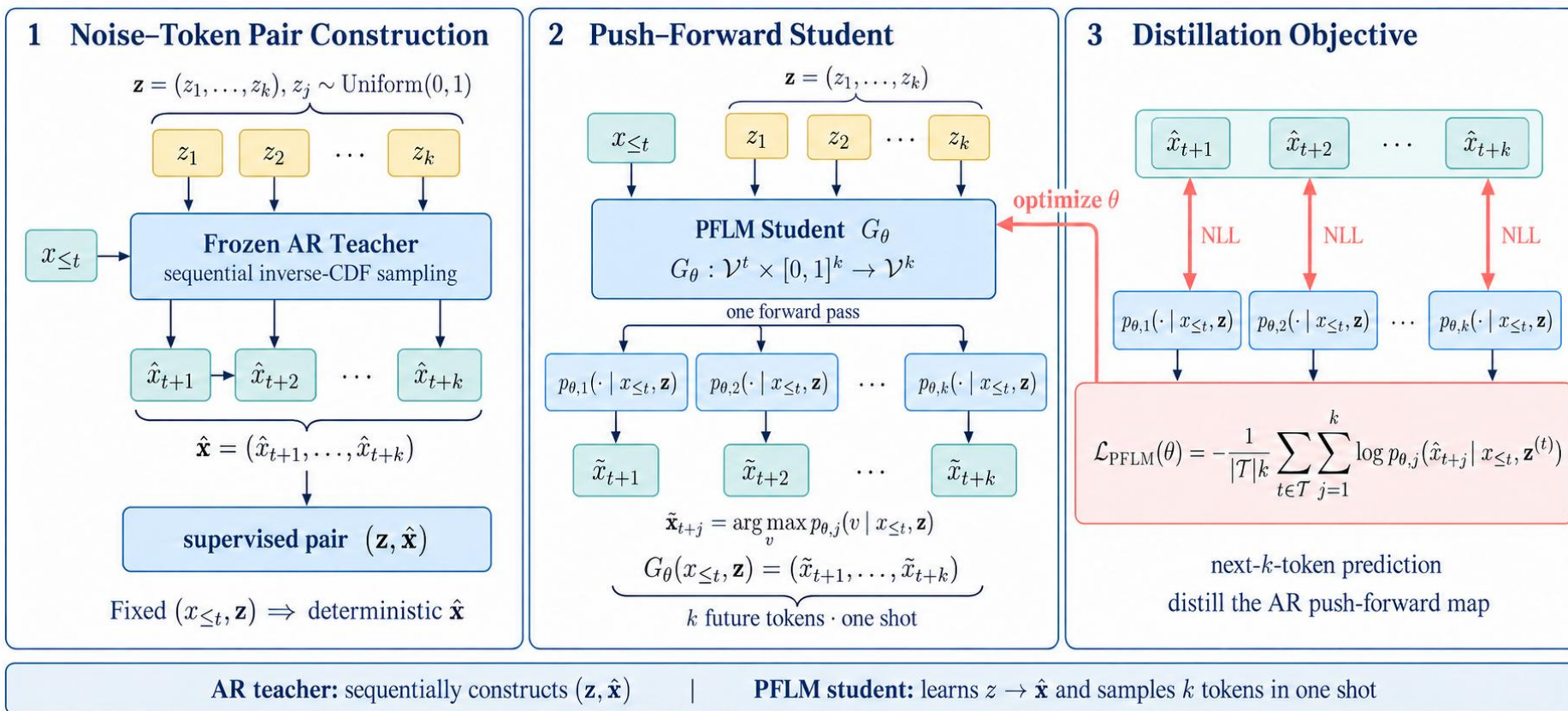


Solution: (a) K-Forcing uses a push-forward language model to map i.i.d. uniform noise tokens to a **fixed-length** block of future tokens, modeling their joint distribution.





Formulation of Push-Forward Language Model



K-Forcing uses **distillation** because the desired push-forward map is **implicitly defined** by the AR teacher's sequential sampling process; teacher-generated noise-token pairs turn this implicit stochastic process into a deterministic supervised mapping

How can we construct noise-token pairs $(\mathbf{z}, \mathbf{x})$ from an AR teacher as ideally as possible?





Baseline: Noise Inversion

Context: The sky is ...

AR teacher distribution	
blue	0.9800
gray	0.0195
green	0.0005

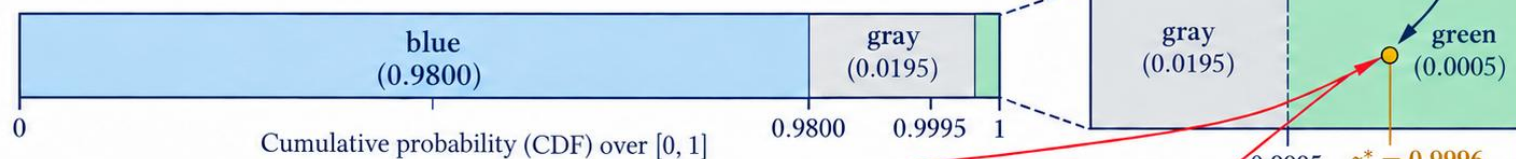
Observed token: **green**

$$l_t = \sum_{v < x_{t+1}} q_{\text{AR}}(v | x_{\leq t}) \quad u_t = l_t + q_{\text{AR}}(x_{t+1} | x_{\leq t})$$

$$[l_t, u_t] = [0.9995, 1]$$

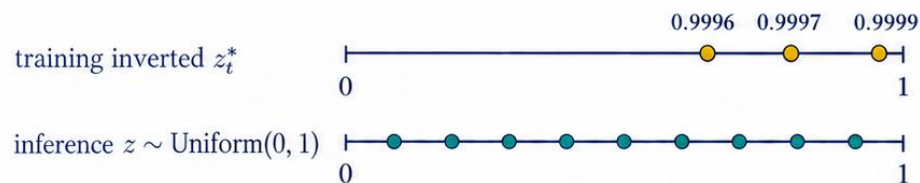
$$z_t^* = 0.9996$$

$$z_t^* \sim \text{Uniform}(l_t, u_t)$$



$$F_q^{-1}(z_t^*) = \text{green}$$

Drawback 1 · Train-inference mismatch



Rare real-data targets concentrate z^* in narrow CDF bins
(this example shows a low-probability token)

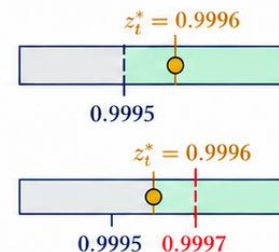
Drawback 2 · Numerical fragility

Original boundary:

green begins at 0.9995 $\rightarrow z^* = 0.9996 \rightarrow$ green ✓

After a small perturbation (boundary + 0.0002):

green begins at 0.9997 $\rightarrow z^* = 0.9996 \rightarrow$ gray ✗

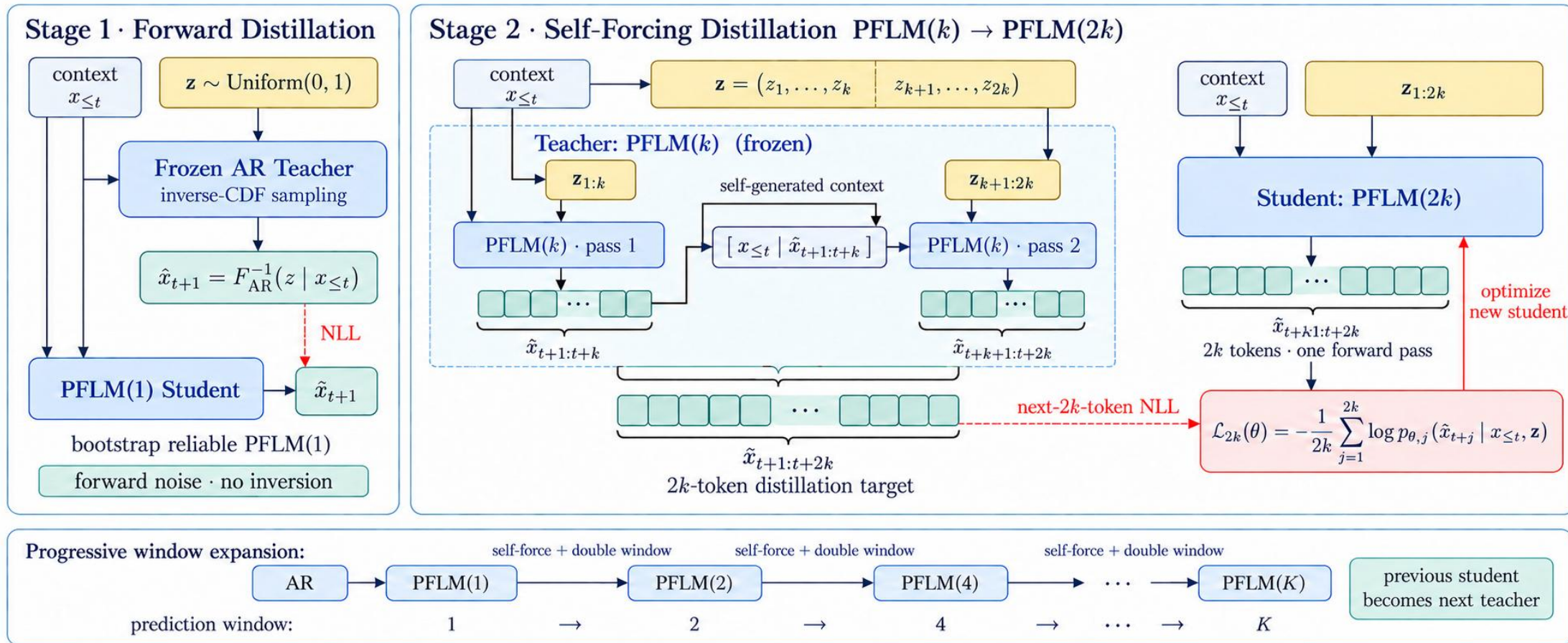


Tiny logit / cumulative-sum changes can cross a narrow bin boundary

Numerical fragility arises from **extremely narrow CDF bins**, **non-bitwise-reproducible** cuBLAS computations, and **non-deterministic floating-point accumulation** in attention and cumulative-sum kernels.



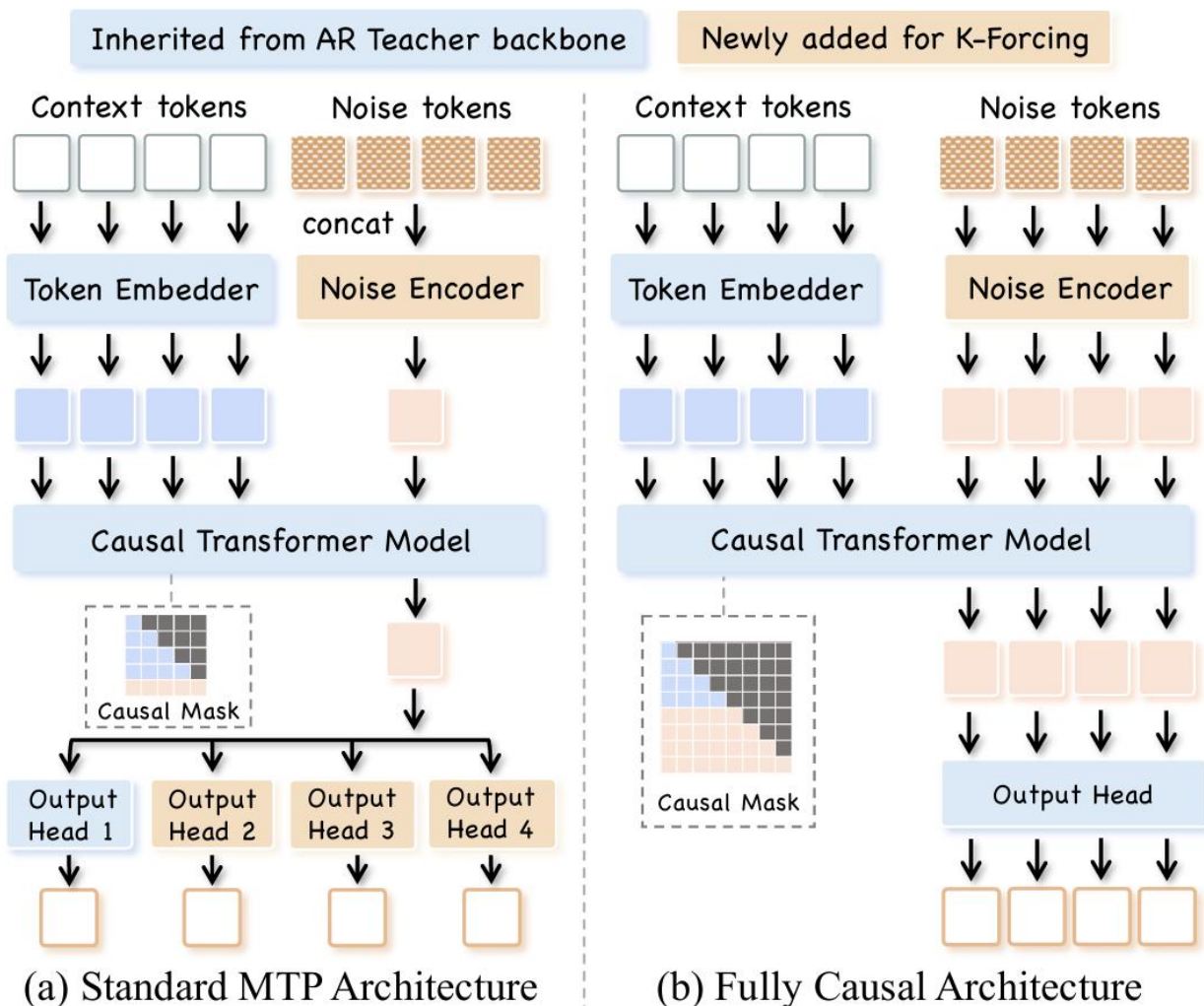
K-Forcing: Progressive Self-Forcing Distillation



This construction removes the train–inference mismatch because the noise is uniform by construction, and avoids recovering noise from narrow CDF bins.



Architecture Design



(a) compress all noise variables into **one** conditioning token and decode k future positions with **independent heads**

(b) represent each noise variable as a **separate causal token**, so the j-th prediction depends only on context and $z_{1:j}$





- **Five Parallelism Strategies**
- **Vanilla Speculative Decoding**
 - **EAGLE families as representative of draft-then-verify paradigm**
 - **DFlash, DSpark introduce diffusion-LLM in Multiple-token prediction**
 - **K-forcing is more batch-serving friendly**

